A

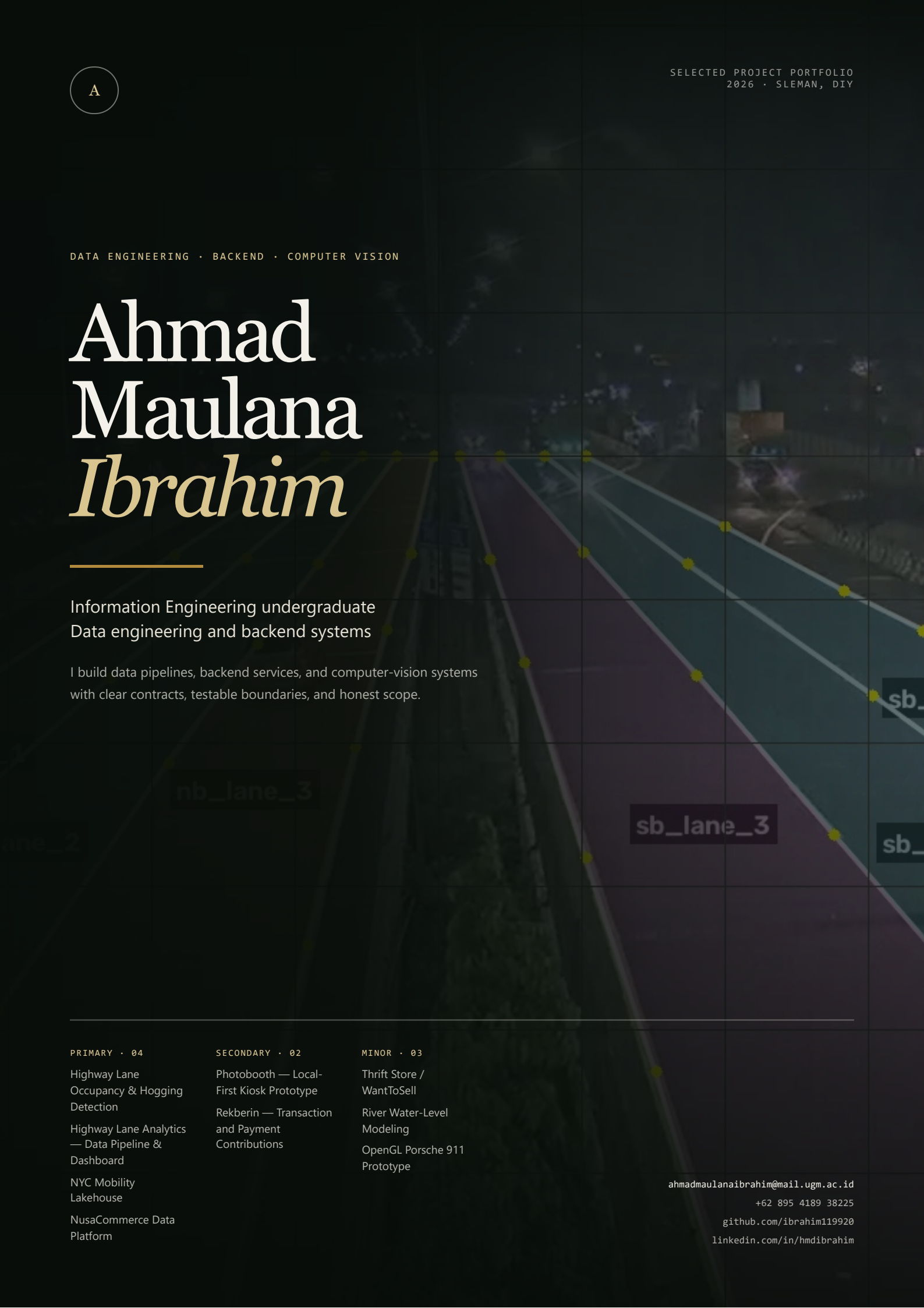
SELECTED PROJECT PORTFOLIO
2026 · SLEMAN, DIY

DATA ENGINEERING · BACKEND · COMPUTER VISION

Ahmad Maulana *Ibrahim*

Information Engineering undergraduate
Data engineering and backend systems

I build data pipelines, backend services, and computer-vision systems with clear contracts, testable boundaries, and honest scope.



PRIMARY · 04

- Highway Lane Occupancy & Hogging Detection
- Highway Lane Analytics — Data Pipeline & Dashboard
- NYC Mobility Lakehouse
- NusaCommerce Data Platform

SECONDARY · 02

- Photobooth — Local-First Kiosk Prototype
- Rekberin — Transaction and Payment Contributions

MINOR · 03

- Thrift Store / WantToSell
- River Water-Level Modeling
- OpenGL Porsche 911 Prototype

ahmadmaulanaibrahim@mail.ugm.ac.id
+62 895 4189 38225
github.com/ibrahim119920
linkedin.com/in/hmdibrahim

PROFILE

Data systems, *built with care.*

3.41
GPA · SCALE NOT STATED
2024 — EXPECTED 2028

Information Engineering undergraduate at Universitas Gadjah Mada. My main focus is data pipelines and backend systems; machine learning and cloud architecture are supporting areas in project work.

WHAT I WORK ON

Pipelines, contracts, and systems that can be checked.

My project work sits where data engineering meets backend software: ingestion and incremental loads, medallion models, event contracts, transactional APIs, data-quality checks, and automated tests.

I prefer explicit boundaries over hidden coupling. The highway case study, for example, separates a vision pipeline that only emits events from a data platform that owns Kafka, storage, and the dashboard. That makes both halves testable in isolation.

EDUCATION

University	**Universitas Gadjah Mada**
Program	**Bachelor of Engineering, Information Engineering**
Study period	**2024 — expected 2028**
Location	**Sleman, DIY**

ORGANIZATIONAL EXPERIENCE

Find IT! UGM · Oct 2024 — May 2025	**Staff, Event Division**
Enforian UGM · Sep — Dec 2024	**Staff, Logistics & Catering**

AT A GLANCE

09 SELECTED PROJECTS	04 PRIMARY PROJECTS
30M+ TRIPS PROCESSED	109 AUTOMATED TESTS

DATA ENGINEERING

`Python` `SQL` `PySpark` `Pandas` `PyArrow` `Parquet` `Kafka` `DuckDB` `Delta Lake` `Databricks`

BACKEND & DATABASES

`FastAPI` `Next.js` `Express` `PostgreSQL` `SQLite` `MongoDB` `SQLAlchemy` `Prisma`

VISION & ML

`YOLO11n` `ByteTrack` `OpenCV` `scikit-learn` `CatBoost` `XGBoost` `PyTorch`

INFRASTRUCTURE & TESTING

`Docker Compose` `GitHub Actions` `pytest` `Newman` `Streamlit` `Plotly`

PRIMARY

01 Highway Lane Occupancy & Hogging Detection

02 Highway Lane Analytics — Data Pipeline & Dashboard

03 NYC Mobility Lakehouse

04 NusaCommerce Data Platform

SECONDARY

01 Photobooth — Local-First Kiosk Prototype

02 Rekberin — Transaction and Payment Contributions

MINOR & COURSEWORK

01 Thrift Store / WantToSell — Backend and Testing

02 River Water-Level Modeling — TMA Experiment Family

03 OpenGL Porsche 911 Prototype

PRIMARY PROJECTS / 01–02

One event contract, *two systems.*

A vision pipeline turns highway CCTV into typed lane events. A Kafka and DuckDB platform moves those events from webhook to dashboard.

CCTV → YOLO11n + ByteTrack → lane mapper → events.jsonl → webhook → FastAPI → Kafka → DuckDB / Parquet → Streamlit

01 · COMPUTER VISION

Highway Lane Occupancy & Hogging Detection

Single-camera prototype · prerecorded video · CPU-first · six calibrated lanes

github.com/ibrahim119920/highway-lane-analytics

Maps highway CCTV vehicles to six normalized lane polygons with YOLO11n and ByteTrack, then emits typed lane events and rule-based lane-hogging indications with snapshot and eight-second clip evidence.

- Normalized polygons keep one calibration profile valid across resolutions; lane 3 carries the overtaking role in both directions.
- Adaptive anchor moves from box centre to the outer bottom corner for wide near-field vehicles; hysteresis prevents boundary flicker from creating false lane changes.
- Rules are configuration: heavy vehicles in an overtaking lane and prolonged car dwell produce severity-tagged events, not legal determinations.

158	111	8.65	109
VEHICLES TRACKED	EVENTS · 5 MIN CLIP	FPS ON CPU	AUTOMATED TESTS

Python · YOLO11n · ByteTrack · OpenCV · pytest

02 · STREAMING DATA

Highway Lane Analytics — Data Pipeline & Dashboard

FastAPI receiver · Kafka · DuckDB / Parquet medallion · Streamlit · local prototype

highway-lane-analytics-data-pipeline · private repository

A replay-fed data platform that receives the vision project's event batches, validates and de-duplicates them, publishes to Kafka, and builds bronze, silver, and gold layers for a four-page Streamlit dashboard.

- FastAPI receiver with bearer-token auth, batch limits, deterministic event keys, and de-duplication before publishing.
- Three Kafka topics keyed by camera_id; consumer quarantines invalid records and commits offsets only after a successful write.
- Silver facts for runs, events, tracks, and lane stints; gold aggregates lane usage, class mix, hogging, transitions, and run quality. Replay is idempotent.

111	158	0	4
EVENTS INGESTED	TRACKS UPSERTED	QUARANTINE	DASHBOARD PAGES

FastAPI · Kafka · DuckDB · Parquet · Streamlit · pytest

Scope note. Prototype on one camera profile and two short clips. Replay is not live inference; lane hogging is a rule-based indication, not a violation; vehicle counts are a lower bound because track IDs can switch after occlusion.

PRIMARY PROJECTS / 03–04

Batch pipelines with *provable behavior.*

Two data-engineering builds focused on idempotency, explicit data contracts, reconciliation, and reproducible local runs.

03 · LAKEHOUSE

NYC Mobility Lakehouse

Databricks Free Edition · NYC TLC Yellow Taxi · Jan–Sep 2024 · Bronze–Silver–Gold

github.com/ibrahim119920/nyc-mobility-lakehouse

An idempotent Bronze → Silver → Gold batch pipeline for 30,020,554 NYC TLC Yellow Taxi trips across nine months, built with PySpark, SQL, and Delta Lake.

- One four-task Job with a replaceWhere overwrite per source_month; the March–September backfill reused the same Job rather than a separate notebook.
- Idempotency is evidenced by Delta version history and identical checksums before and after a rerun.
- Gold is reconciled against Silver with zero difference, then used for 12 descriptive analysis queries covering monthly, daily, hourly, zone, payment, and duration patterns.

30.0M	9	12	0
TRIPS PROCESSED	MONTHS LOADED	ANALYSIS QUERIES	RECONCILIATION GAP

`Databricks` `PySpark` `SQL` `Delta Lake` `Medallion` `replaceWhere`

04 · DATA PLATFORM

NusaCommerce Data Platform

Architecture and incremental ingestion · synthetic e-commerce data · local runner

github.com/ibrahim119920/nusacommerce-data-platform

Designed a data platform for NusaCommerce: PostgreSQL read replica and Payment REST API sources, a Bronze → Silver → Gold → marts → dashboard flow, quality contracts, and freshness SLOs.

- Incremental batch ingestion with a composite watermark, keyset pagination, safe cutoff, 15-minute lookback, and checkpoints that advance only after a durable commit.
- Implementation covers per-page staging, atomic finalization, and rerun recovery; the current running path is orders ingestion from a local table with one sequential runner.
- Quality and freshness are treated as part of the contract, not a post-hoc report.

3	2	15	1
SOURCES DESIGNED	DATA CONTRACTS	MIN LOOKBACK	ACTIVE RUNNER

`Python` `PostgreSQL` `SQLAlchemy` `Data architecture` `Data quality` `Docker Compose`

Scope notes. NYC covers January–September 2024 only; October–December are not loaded, so no full-year claim is made. NusaCommerce's Payment API path remains an exercise; dbt, Airflow, Kafka, and clickstream are not implemented yet. Analyses are descriptive, not causal.

SECONDARY PROJECTS / 01–02

Boundaries that make *operations testable.*

One local-first kiosk with a hardware boundary; one team marketplace with transactional payment flows.

01 · LOCAL-FIRST PROTOTYPE

Photobooth — Local-First Kiosk Prototype

Next.js · SQLite · C++ camera bridge · single device, single active session

Private repository

A kiosk prototype where Next.js serves the UI and local backend, SQLite and a local folder are the operational source of truth, and a Local Camera Service is the integration boundary to a USB mirrorless camera.

- Session state machine with one active session locked in the database, idempotent commands, transactions, audit logs, and local health/backup checks.
- C++ camera bridge boundary with fake-bridge tests, so the capture flow can be tested without physical hardware.
- Result media is stored locally before a durable upload queue hands it to cloud storage.

Choose package → QRIS + operator PIN → camera capture → local save → Drive queue → result QR

1	SQLite	C++	Fake
ACTIVE SESSION	LOCAL SOURCE OF TRUTH	CAMERA BOUNDARY	BRIDGE TESTS

Next.js · SQLite · C++ bridge · Transactions · Audit logs · SSE

02 · TEAM BACKEND

Rekberin — Transaction and Payment Contributions

Public team repository · individual contributions via PRs #28, #38, #44

github.com/aufaanggara/rekberin

Contributed persisted transaction APIs and payment integration to a game-account marketplace with negotiation, QRIS payment, and a handover flow.

- TypeScript, Next.js, Prisma, and PostgreSQL transaction APIs with atomic listing reservations and participant-based access control.
- Midtrans Core API sandbox integration for QRIS, webhook signature verification, and payment reconciliation.
- Transaction and payment tests added through merged pull requests #28, #38, and #44.

Listing → offer & chat → QRIS payment → webhook → handover → payout simulation

3	QRIS	2 h	Atomic
MERGED PULL REQUESTS	MIDTRANS SANDBOX	BUYER CONFIRMATION	LISTING RESERVATION

TypeScript · Next.js · Prisma · PostgreSQL · Midtrans · REST APIs

Scope notes. Photobooth's physical camera operation was unverified in the document snapshot; testing used a fake bridge. Rekberin's Midtrans integration uses the sandbox; seller payouts and refunds are simulated.

MINOR PROJECTS & COURSEWORK / 01–03

Smaller builds, *same engineering habits.*

Shorter project work that still shows transactional thinking, experiment discipline, and low-level rendering.

01 · Backend & testing

Thrift Store / WantToSell

Express and MongoDB backend for a thrift marketplace with catalog, favorites, orders, reviews, and transactional stock-one reservations.

- One item per product reserved transactionally, with order expiry and simulated payment.
- A recorded ten-request purchase race produced one order and nine conflicts.
- API and integrity tests with Postman/Newman; the snapshot records one failed assertion.

JavaScript · Express · MongoDB · Mongoose · Newman

github.com/ibrahim119920/web-thrift-store

02 · ML experiments

River Water-Level Modeling

A TMA forecasting experiment family covering seasonal baselines, residual boosting ensembles, temporal validation, and experiment provenance.

- Station-aware and upstream-river features; a supermodel orchestrator selects a model family per station.
- PyTorch notebooks cover graph/time-series neural model training and inference.
- 30-station registry and seven model families; training used two Kaggle T4 GPUs.

CatBoost · XGBoost · PyTorch · Pandas · Time series

github.com/ibrahim119920/TMA-CatBoostRegressor

03 · Graphics

OpenGL Porsche 911

A C++/OpenGL rendering prototype for a Porsche 911 model, focused on mesh and shader rendering plus model loading.

- Mesh/model rendering pipeline with vertex and fragment shaders.
- Model loading through Assimp; windowing and input through GLFW.
- Math and transform work through GLM; GLAD provides the OpenGL function loader.

C++ · OpenGL · GLFW · GLAD · GLM · Assimp

Private repository

LET'S BUILD RELIABLE SYSTEMS

Clear contracts. *Honest scope.*

I am interested in data engineering, backend systems, and applied computer vision. The work above is project-based; where a claim is limited by data, hardware, or environment, the scope note says so.

Data pipelines · Backend APIs · Computer vision · Data quality

ahmadmaulanaibrahim@mail.ugm.ac.id
+62 895 4189 38225
github.com/ibrahim119920
linkedin.com/in/hmdibrahim
Sleman, DIY · Indonesia

SELECTED PROJECT PORTFOLIO · 2026